

# Auditoría Estática – Fiabilidad (ISO/IEC 25010)

**Sistema auditado:** Sistema de Gestión Bibliotecaria – Equipo 58-1

**Auditor:** Equipo SQA 11

**Fecha de auditoría:** 2 de junio de 2026

**Estándar base:** ISO/IEC 25010:2023 – Característica *Reliability* (Tolerancia a Fallos y Capacidad de Recuperación)

**Guía metodológica:** ISO/IEC/IEEE 29119 – Pruebas Estáticas

**Framework técnico auditado:** Java 21 / Spring Boot 3.4.5 / Vue 3.2.13

**Versión del checklist aplicado:** 2.1 (con columna Dimensión)

**Modalidad:** Revisión en frío, individual e independiente (*cold review*)

## 1. Resumen Ejecutivo

Se aplicó la checklist especializada de **Fiabilidad** al repositorio de código del Sistema de Gestión Bibliotecaria del Equipo 58-1. La evaluación comprendió los **11 ítems** distribuidos en las dos sub-características del estándar:

- **Tolerancia a Fallos** (ítems 1 a 6)
- **Capacidad de Recuperación** (ítems 7 a 11)

### Veredicto del Quality Gate: FAIL

| Métrica                  | Valor                                       |
|--------------------------|---------------------------------------------|
| Total de ítems evaluados | 11                                          |
| Cumple                   | 0                                           |
| No Cumple                | 7                                           |
| No Aplica                | 4                                           |
| Defectos detectados      | 12 (3 CRITICAL · 4 HIGH · 3 MEDIUM · 2 LOW) |

El Quality Gate del pipeline (sección 3 del checklist) exige **cero ocurrencias de "No Cumple"** en tolerancia a fallos y capacidad de recuperación como condición necesaria para transicionar hacia las fases de pruebas dinámicas. La presente auditoría **no satisface** dicho criterio.

## Distribución por dimensión

| Dimensión             | Cumple | No Cumple | No Aplica |
|-----------------------|--------|-----------|-----------|
| Backend (Java)        | 0      | 4         | 2         |
| Frontend (Vue)        | 0      | 2         | 0         |
| Transversal           | 0      | 1         | 2         |
| Arquitectura de Datos | 0      | 1         | 0         |
| Total                 | 0      | 7         | 4         |

## 2. Tabla Consolidada de Resultados

| ID          | Dimensión             | Ítem de Verificación                                                                   | Estado    | Evidencia principal                                                      |
|-------------|-----------------------|----------------------------------------------------------------------------------------|-----------|--------------------------------------------------------------------------|
| COD-FIAB-01 | Backend (Java)        | Captura de excepciones específicas en operaciones críticas (IO, parsing de fechas, DB) | No Cumple | <code>PrestamoService.java:47</code>                                     |
| COD-FIAB-02 | Backend (Java)        | Timeouts explícitos en llamadas síncronas a servicios externos                         | No Aplica | Sin clientes HTTP salientes                                              |
| COD-FIAB-03 | Transversal           | Patrones de resiliencia (Circuit Breaker) ante acoplamientos remotos                   | No Aplica | Sin dependencias remotas                                                 |
| COD-FIAB-04 | Backend (Java)        | Manejador global de excepciones ( <code>@ControllerAdvice</code> )                     | No Cumple | Ausencia total en el backend                                             |
| COD-FIAB-05 | Frontend (Vue)        | Manejador global de errores de renderizado ( <code>onErrorCaptured</code> )            | No Cumple | <code>main.js:1-4</code>                                                 |
| COD-FIAB-06 | Transversal           | Manejo de fallos en operaciones asíncronas                                             | No Cumple | Múltiples componentes Vue                                                |
| COD-FIAB-07 | Backend (Java)        | Contexto transaccional atómico en mutaciones múltiples                                 | No Cumple | <code>PrestamoService.java:58,61,79,83,96,149,154</code>                 |
| COD-FIAB-08 | Arquitectura de Datos | Parámetros del pool de conexiones para reestablecer enlaces caídos                     | No Cumple | <code>application.properties</code> (sin claves <code> Hikari.*</code> ) |

| ID          | Dimensión             | Ítem de Verificación                                                              | Estado           | Evidencia principal                           |
|-------------|-----------------------|-----------------------------------------------------------------------------------|------------------|-----------------------------------------------|
| COD-FIAB-09 | Backend (Java)        | Políticas de reintento ( <i>Retry</i> ) con retroceso exponencial                 | No Aplica        | Sin peticiones distribuidas                   |
| COD-FIAB-10 | Frontend (Vue)        | Estados de degradación elegante ( <i>fallback</i> ) ante indisponibilidad del API | <b>No Cumple</b> | Sin interceptores axios ni manejo 5xx         |
| COD-FIAB-11 | Arquitectura de Datos | Bitácora de tareas programadas para reanudación segura                            | No Aplica        | Sin <code>@Scheduled</code> ni procesos batch |

## 3. Detalle de Evaluación por Ítem

### 3.1 COD-FIAB-01 – Captura de excepciones específicas

**Dimensión:** Backend (Java)

**Pregunta:** ¿El código captura excepciones específicas en operaciones críticas (IO, parsing de fechas, DB) evitando bloques `catch` genéricos vacíos?

**Veredicto:** No Cumple

**Evidencia:**

```
src/main/java/com/biblioteca/service/PrestamoService.java:47:
```

```
    LocalDate fechaPrestamo = LocalDate.parse(fechaPrestamoStr);
```

Parsing directo de fecha sin bloque `try/catch`. Una entrada con formato inválido propaga

`DateTimeParseException` sin control.

```
src/main/java/com/biblioteca/service/LibroService.java:48-52:
```

```
try {
    libro.setImagen(imagen.getBytes());
} catch (Exception e) {
    return "Error al procesar la imagen.";
}
```

Único bloque `catch` del backend. Utiliza `Exception` genérica en lugar de `IOException` específica, y oculta la excepción sin registrarla en bitácora.

`src/main/java/com/biblioteca/controller/Controller.java` (archivo completo, 458 líneas): no contiene un solo bloque `try/catch` en ninguno de sus endpoints.

**Análisis:** El defecto original **COD-13** ("PrestamoService hace parsing directo sin control") **no fue remediado**. La línea 47 de `PrestamoService.java` realiza `LocalDate.parse()` directamente sobre el string recibido en el DTO de entrada, sin validación previa ni captura de `DateTimeParseException`. Esto deriva en respuestas HTTP 500 con stacktrace potencialmente expuesto ante entradas malformadas.

---

## 3.2 COD-FIAB-02 – Timeouts en llamadas a servicios externos

**Dimensión:** Backend (Java)

**Pregunta:** ¿Las llamadas síncronas a servicios externos o APIs de terceros implementan políticas explícitas de *timeout* de conexión y lectura?

**Veredicto:** No Aplica

**Evidencia:**

- Búsqueda exhaustiva de `RestTemplate`, `WebClient`, `FeignClient`, `HttpClient`: 0 ocurrencias en el proyecto.
- `pom.xml` declara únicamente `spring-boot-starter-web`, `spring-boot-starter-data-jpa`, `spring-boot-starter-security`, `spring-boot-starter-test`, `mysql-connector-java` y `lombok`. Ningún cliente HTTP externo.

**Análisis:** La aplicación es un monolito que sólo se comunica con su base de datos MySQL local mediante JPA/Hibernate. No existen acoplamientos HTTP salientes que requieran configuración de *timeout*.

---

## 3.3 COD-FIAB-03 – Circuit Breaker / patrones de resiliencia

**Dimensión:** Transversal

**Pregunta:** ¿Se utilizan patrones de resiliencia tipo *Circuit Breaker* o aislamiento ante fallos en acoplamientos remotos?

**Veredicto:** No Aplica

**Evidencia:**

- Búsqueda de `@CircuitBreaker`, `Resilience4j`, `HystrixCommand`: 0 ocurrencias.

- `pom.xml` no contiene dependencias `resilience4j-*` ni `spring-cloud-circuitbreaker`.
- No existen acoplamientos remotos (confirmado en COD-FIAB-02).

**Análisis:** Al no existir superficie de fallo distribuida, el ítem carece de objeto de auditoría.

---

## 3.4 COD-FIAB-04 – Manejador global de excepciones

**Dimensión:** Backend (Java)

**Pregunta:** ¿El backend dispone de un manejador global de excepciones para interceptar errores imprevistos, evitando caídas completas del hilo y fugas de `stacktrace`?

**Veredicto:** No Cumple

**Evidencia:**

- Búsqueda exhaustiva de `@ControllerAdvice`, `@RestControllerAdvice`, `@ExceptionHandler`, `HandlerExceptionResolver`, `ErrorController`: 0 ocurrencias en todo el proyecto.
- `src/main/java/com/biblioteca/config/SecurityConfig.java` (única clase de configuración, 55 líneas) sólo configura `SecurityFilterChain` y `PasswordEncoder`.
- No existe ninguna clase bajo el namespace `com.biblioteca` con nomenclatura `*ExceptionHandler`, `*ErrorHandler` o `*GlobalException`.

**Análisis:** El backend carece de un mecanismo centralizado de manejo de excepciones. Cuando un service lanza `RuntimeException` (por ejemplo en `LibroService.java:33`, `LibroService.java:74`, `UsuarioService.java:95`, `ResenaService.java:35`, `ComentarioResenaService.java:39`) o se produce un `DateTimeParseException` (COD-FIAB-01), la excepción propaga hasta el contenedor de Spring, generando una respuesta HTTP 500 con `stacktrace` completo potencialmente expuesto al cliente. Esto constituye una fuga de información interna y una experiencia de error no controlada.

---

## 3.5 COD-FIAB-05 – Manejador global de errores de renderizado (Vue)

**Dimensión:** Frontend (Vue)

**Pregunta:** En la interfaz de usuario (Vue 3), ¿se hace uso del *hook* de ciclo de vida `onErrorCaptured` o de un manejador global de errores para encapsular fallos de renderizado?

**Veredicto:** No Cumple

**Evidencia:**

```
biblioteca-frontend/src/main.js:1-4:
```

```
import { createApp } from 'vue'
import App from './App.vue'

createApp(App).mount('#app')
```

No se configura `app.config.errorHandler`. La aplicación se monta sin interceptor de errores global.

Búsqueda de `onErrorCaptured`, `errorCaptured`, `errorHandler`, `window.onerror`, `window.addEventListener('error', ...)` : 0 ocurrencias en todos los archivos `.vue` y `.js` del proyecto.

**Análisis:** Ningún componente Vue ni el punto de entrada de la aplicación implementan manejo global de errores de renderizado. Si ocurre un error de JavaScript durante el renderizado de un componente (por ejemplo, acceso a propiedades de un objeto `null` o `undefined`), la aplicación fallará silenciosamente o mostrará un error en consola sin retroalimentación al usuario, dejando la UI en un estado inconsistente.

---

## 3.6 COD-FIAB-06 – Manejo de fallos en operaciones asíncronas

**Dimensión:** Transversal

**Pregunta:** ¿Las operaciones asíncronas (`@Async` en Spring / *Promises* en Vue 3) poseen su respectivo manejo interno de fallos y rechazos?

**Veredicto:** No Cumple

**Evidencia – Backend:**

- Búsqueda de `@Async` : 0 ocurrencias. El backend no implementa operaciones asíncronas, por lo que el ítem no aplica en este dominio.

**Evidencia – Frontend (llamadas asíncronas sin manejo de errores):**

```
biblioteca-frontend/src/components/PantallaLibro.vue:233-239 –
agregarResena :
```

```
await axios.post('/api/resenas', {
  libroId: props.libro.id,
  usuarioId: usuarioActual.value.id,
  texto: nuevaResena.value
```

```
  })
  nuevaResena.value = ''
  await cargarResenas()
```

biblioteca-frontend/src/components/PantallaLibro.vue:242-245 –  
eliminarResena:

```
  await axios.delete(`/api/resenas/${resenaId}`)
  await cargarResenas()
```

Adicionalmente, en el mismo archivo: `guardarEdicionResena` (líneas 252-259),  
`agregarComentarioResena` (266-280), `eliminarComentarioResena` (282-285) y  
`guardarEdicionComentario` (292-299) presentan la misma carencia.

biblioteca-frontend/src/components/SolicitudVerificacionPago.vue:69-83 –  
pagarAmonestacion:

```
const pagarAmonestacion = async (amonestacionId) => {
  await fetch('/api/amonestaciones-usuario/pagar', { /* ... */ });
  await cargarAmonestaciones();
  // ...
};
```

biblioteca-frontend/src/components/VerificarPago.vue:43-47 –  
cargarAmonestaciones:

```
const cargarAmonestaciones = async () => {
  const res = await fetch('/api/amonestaciones-usuario/todas', { credenti
  const data = await res.json();
  amonestaciones.value = data || [];
};
```

Adicionalmente, `verificarAmonestacion` (líneas 49-55) en el mismo archivo carece de  
`try/catch`.

**Análisis:** Aunque el backend no implementa `@Async`, el frontend contiene múltiples operaciones  
asíncronas (`axios` y `fetch`) sin manejo de rechazos. Se identificaron **al menos 9 funciones  
asíncronas** en `PantallaLibro.vue`, `SolicitudVerificacionPago.vue` y  
`VerificarPago.vue` que no envuelven sus llamadas en `try/catch` ni encadenan `.catch()`.

Un fallo de red o de servidor generará *unhandled promise rejections*, pudiendo romper el estado de la UI o causar comportamientos indefinidos.

---

### 3.7 COD-FIAB-07 – Transacciones atómicas en mutaciones múltiples

**Dimensión:** Backend (Java)

**Pregunta:** ¿Todas las operaciones de mutación múltiple en el almacenamiento (persistencia) se ejecutan bajo un contexto transaccional atómico controlado?

**Veredicto:** No Cumple

**Evidencia – Único uso correcto de `@Transactional` :**

`src/main/java/com/biblioteca/service/UsuarioService.java:92-119 – eliminarUsuario :`

```
@Transactional
public void eliminarUsuario(String correo) {
    // ... deleteAll(comentarios), deleteAll(resenas), delete(usuario)
}
```

Tres operaciones de borrado correctamente protegidas.

**Evidencia – Métodos con mutaciones múltiples SIN `@Transactional` :**

`src/main/java/com/biblioteca/service/PrestamoService.java:29-64 – crearPrestamo :`

```
prestamoRepository.save(prestamo);    // línea 58
// ...
libroRepository.save(libro);           // línea 61
```

Dos operaciones de escritura en entidades diferentes. Si `libroRepository.save()` falla, el préstamo queda persistido sin decremento de stock.

`src/main/java/com/biblioteca/service/PrestamoService.java:67-100 – devolverPrestamo :`

```
prestamoRepository.save(prestamo);    // línea 79
libroRepository.save(libro);           // línea 83
amonestacionRepository.save(amonestacion); // línea 96 (condicional)
```

Hasta tres operaciones de escritura. Si el `save` del libro o de la amonestación falla, el préstamo queda marcado como devuelto pero el stock no se restaura o la amonestación no se registra.

`src/main/java/com/biblioteca/service/PrestamoService.java:118-157` –  
`renovarPrestamo` :

```
prestamoRepository.save(prestamo); // línea 149
libroRepository.save(libro);       // línea 154
```

`src/main/java/com/biblioteca/controller/Controller.java:281-305` –  
`eliminarResena` :

```
comentarioResenaService.eliminarPorResena(resena); // línea 300
resenaService.eliminar(resena);                  // línea 302
```

**Análisis:** Sólo 1 de los 5+ métodos con mutaciones múltiples utiliza `@Transactional` . Los tres métodos críticos del flujo de préstamos ( `crearPrestamo` , `devolverPrestamo` , `renovarPrestamo` ) modifican al menos dos entidades –*Préstamo* y *Libro*, y potencialmente *Amonestación*– sin garantía de atomicidad. Esto puede derivar en estados inconsistentes en la base de datos ante fallos parciales. Adicionalmente, `UsuarioService.java:16` importa `jakarta.transaction.Transactional` en lugar de `org.springframework.transaction.annotation.Transactional` , lo cual funciona pero no habilita atributos específicos de Spring como `rollbackFor` .

---

## 3.8 COD-FIAB-08 – Configuración del Pool de Conexiones

**Dimensión:** Arquitectura de Datos

**Pregunta:** ¿La configuración del Pool de Conexiones a la Base de Datos posee parámetros estáticos para reestablecer enlaces caídos de forma segura?

**Veredicto:** No Cumple

**Evidencia:**

`src/main/resources/application.properties` (archivo completo, 22 líneas):

```
spring.datasource.url=jdbc:mysql://localhost:3306/biblioteca_db
spring.datasource.username=root
spring.datasource.password=admin
```

```
spring.jpa.show-sql=true
spring.jpa.generate-ddl=true
spring.jpa.hibernate.ddl-auto=update
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
```

No existe ninguna propiedad `spring.datasource.hikari.*`. Búsqueda de `hikari`, `connectionTimeout`, `idleTimeout`, `maxLifetime`, `testOnBorrow`: 0 ocurrencias.

**Análisis:** Spring Boot utiliza HikariCP por defecto como pool de conexiones, pero no se configura ningún parámetro explícito. Si MySQL cierra conexiones inactivas (timeout por defecto: 8 horas), HikariCP podría intentar reutilizar conexiones muertas, generando errores `Communications link failure` sin mecanismo de recuperación configurado.

---

### 3.9 COD-FIAB-09 – Políticas de Retry con retroceso exponencial

**Dimensión:** Backend (Java)

**Pregunta:** ¿Las peticiones distribuidas propensas a fallas transitorias de red implementan políticas estáticas de reintento (Retry) con retroceso exponencial?

**Veredicto:** No Aplica

**Evidencia:**

- Búsqueda de `@Retryable`, `RetryTemplate`, `spring-retry`, `resilience4j-retry`: 0 ocurrencias.
- `pom.xml` no contiene dependencias de `retry`.
- No existen peticiones distribuidas en el código (confirmado en COD-FIAB-02 y COD-FIAB-03).

**Análisis:** La única comunicación externa es con la base de datos MySQL local mediante JPA, donde la política de reintentos es responsabilidad del pool y del driver JDBC. El ítem carece de objeto de auditoría.

---

### 3.10 COD-FIAB-10 – Fallback / degradación elegante ante indisponibilidad del API

**Dimensión:** Frontend (Vue)

**Pregunta:** ¿La interfaz frontend maneja estados de degradación elegante (*fallback*) amigables frente a la indisponibilidad temporal del servidor API?

**Veredicto:** No Cumple

## Evidencia:

- Búsqueda de interceptores de axios ( `axios.interceptors` ): 0 ocurrencias.
- Búsqueda de manejo explícito de status HTTP 502/503/504: 0 ocurrencias.
- No existe una instancia centralizada de axios. Los componentes que la utilizan ( `PantallaPrincipal.vue` , `PantallaLibro.vue` , `EliminarLibroPantallaBusqueda.vue` , `ModificarLibroPantallaBusqueda.vue` , `ModificarLibroFormulario.vue` ) la importan directamente y gestionan errores de forma ad-hoc.
- `biblioteca-frontend/src/components/SolicitudVerificacionPago.vue:57-67` — `cargarAmonestaciones` presenta `try/finally` pero carece de `catch` , por lo que el error de red no se maneja ni se muestra al usuario.
- `biblioteca-frontend/src/components/VerificarPago.vue:43-47` — `cargarAmonestaciones` carece por completo de `try/catch` .
- Ningún componente implementa una pantalla de "Servicio no disponible" o "Error de conexión" como *fallback*.

**Análisis:** El frontend no posee una estrategia centralizada ni consistente para manejar la indisponibilidad del servidor. Algunos componentes ( `AnadirPrestamo.vue:122-124` , `DevolverPrestamo.vue:165-167` ) muestran mensajes genéricos de error en su `catch` , pero no distinguen entre error de negocio, de autenticación o de servidor caído. No existen interceptores de axios para capturar errores globalmente, ni pantallas de *fallback* para estados 5xx.

---

## 3.11 COD-FIAB-11 – Bitácora de tareas programadas

**Dimensión:** Arquitectura de Datos

**Pregunta:** En procesos *batch* o tareas programadas ( `@Scheduled` ), ¿existe un mecanismo de bitácora o persistencia de estado para reanudación segura ante interrupciones?

**Veredicto:** No Aplica

## Evidencia:

- Búsqueda de `@Scheduled` , `cron` , `fixedRate` , `fixedDelay` : 0 ocurrencias en todo el proyecto.
- `GestionBibliotecariaApplication.java` no incluye la anotación `@EnableScheduling` .

**Análisis:** La aplicación no ejecuta tareas programadas ni procesos *batch*. El ítem carece de objeto de auditoría.

---

## 4. Catálogo de Defectos Detectados

| #  | Severidad | ID Checklist         | Defecto                                                                                                                                                                                                  | Ubicación                                                                                                          |
|----|-----------|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| 1  | CRITICAL  | COD-FIAB-07          | <code>crearPrestamo()</code> realiza 2 <code>save()</code> (préstamo + libro) sin <code>@Transactional</code> . Si el segundo <code>save</code> falla, el préstamo queda creado sin decremento de stock. | <code>PrestamoService.java:58,61</code>                                                                            |
| 2  | CRITICAL  | COD-FIAB-07          | <code>devolverPrestamo()</code> realiza hasta 3 <code>save()</code> (préstamo + libro + amonestación) sin <code>@Transactional</code> . Inconsistencia de datos ante fallo parcial.                      | <code>PrestamoService.java:79,83,96</code>                                                                         |
| 3  | CRITICAL  | COD-FIAB-04          | Ausencia de <code>@ControllerAdvice / @RestControllerAdvice</code> . Las <code>RuntimeException</code> lanzadas por services generan HTTP 500 con <code>stacktrace</code> potencialmente expuesto.       | Todo el backend                                                                                                    |
| 4  | HIGH      | COD-FIAB-01 (COD-13) | <code>LocalDate.parse(fechaPrestamoStr)</code> sin <code>try/catch</code> . Defecto original COD-13 sin remediación.                                                                                     | <code>PrestamoService.java:47</code>                                                                               |
| 5  | HIGH      | COD-FIAB-07          | <code>renovarPrestamo()</code> realiza 2 <code>save()</code> sin <code>@Transactional</code> .                                                                                                           | <code>PrestamoService.java:149,154</code>                                                                          |
| 6  | HIGH      | COD-FIAB-07          | <code>Controller.eliminarResena()</code> ejecuta borrado de comentarios + borrado de reseña sin transacción.                                                                                             | <code>Controller.java:300-302</code>                                                                               |
| 7  | HIGH      | COD-FIAB-06          | 9 funciones <code>async</code> en componentes Vue sin <code>try/catch</code> ni <code>.catch()</code> .                                                                                                  | <code>PantallaLibro.vue</code> ,<br><code>SolicitudVerificacionPago.vue</code> ,<br><code>VerificarPago.vue</code> |
| 8  | HIGH      | COD-FIAB-05          | Ausencia de <code>app.config.errorHandler</code> en <code>main.js</code> y de <code>onErrorCaptured</code> en componentes.                                                                               | <code>main.js:1-4</code>                                                                                           |
| 9  | MEDIUM    | COD-FIAB-08          | Pool HikariCP sin configuración explícita. Sin <code>maxLifetime</code> , <code>connectionTestQuery</code> ni <code>keepaliveTime</code> .                                                               | <code>application.properties</code>                                                                                |
| 10 | MEDIUM    | COD-FIAB-10          | Sin interceptores de axios ni manejo de HTTP 502/503/504. Sin pantallas de <i>fallback</i> .                                                                                                             | Todo el frontend                                                                                                   |

| #  | Severidad | ID Checklist | Defecto                                                                                                                                                                           | Ubicación                                  |
|----|-----------|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------|
| 11 | MEDIUM    | COD-FIAB-10  | <code>SolicitudVerificacionPago.vue:57-67 ( cargarAmonestaciones )</code> tiene <code>try/finally</code> sin <code>catch</code> .                                                 | <code>SolicitudVerificacionPago.vue</code> |
| 12 | LOW       | COD-FIAB-01  | Único <code>catch</code> del backend usa <code>Exception</code> genérica y no registra la excepción en bitácora.                                                                  | <code>LibroService.java:50</code>          |
| 13 | LOW       | COD-FIAB-07  | <code>UsuarioService.java:16</code> importa <code>jakarta.transaction.Transactional</code> al en lugar de <code>org.springframework.transaction.annotation.Transactional</code> . | <code>UsuarioService.java:16</code>        |

## 5. Recomendaciones de Remediación

### 5.1 Prioridad CRITICAL – resolver de forma inmediata

1. **Implementar `@RestControllerAdvice` global** (COD-FIAB-04). Crear una clase `GlobalExceptionHandler` con `@RestControllerAdvice` que intercepte `RuntimeException`, `DateTimeParseException`, `AccessDeniedException`, `MethodArgumentNotValidException` y `Exception` genérica. Retornar respuestas JSON estructuradas con código de error y mensaje amigable, sin exponer *stacktraces*.
2. **Agregar `@Transactional(rollbackFor = Exception.class)` a los métodos críticos de `PrestamoService`** (COD-FIAB-07): `crearPrestamo()`, `devolverPrestamo()` y `renovarPrestamo()`, utilizando la importación de Spring (`org.springframework.transaction.annotation.Transactional`).
3. **Proteger `LocalDate.parse()` con `try/catch`** (COD-FIAB-01 / COD-13). Envolver el parsing de `PrestamoService.java:47` con `try { ... } catch (DateTimeParseException e)` y retornar un mensaje controlado, o validar el formato antes del parsing.

### 5.2 Prioridad HIGH – resolver antes del siguiente sprint

4. **Agregar `try/catch` a todas las funciones `async` sin manejo en Vue** (COD-FIAB-06). Envolver las 9 funciones identificadas en `PantallaLibro.vue`, `SolicitudVerificacionPago.vue` y `VerificarPago.vue` con bloques que muestren mensajes de error al usuario.

5. **Configurar `app.config.errorHandler` en `main.js`** (COD-FIAB-05). Agregar un manejador global que capture errores de renderizado y muestre una notificación al usuario o redirija a una pantalla de error.
6. **Proteger `Controller.eliminarResena()` con transacción** (COD-FIAB-07). Mover la lógica de borrado de reseña + comentarios a un método de servicio con `@Transactional`, o crear un método en `ResenaService` que encapsule ambas operaciones.

## 5.3 Prioridad MEDIUM – planificar para la siguiente iteración

7. **Configurar HikariCP explícitamente** (COD-FIAB-08). Agregar en `application.properties` las siguientes propiedades:

```
spring.datasource.hikari.maximum-pool-size=10
spring.datasource.hikari.minimum-idle=5
spring.datasource.hikari.connection-timeout=20000
spring.datasource.hikari.max-lifetime=1200000
spring.datasource.hikari.keepalive-time=300000
```

8. **Crear interceptor centralizado de axios** (COD-FIAB-10). Configurar una instancia única de axios con interceptor de respuesta que detecte HTTP 502/503/504 y muestre un *toast* o *banner* de "Servicio temporalmente no disponible". Considerar un componente *Error Boundary*.
9. **Corregir `SolicitudVerificacionPago.vue:57-67`** (COD-FIAB-10). Agregar bloque `catch` al `try/finally` existente para manejar errores de carga.

## 5.4 Prioridad LOW – mejora continua

10. **Cambiar `catch (Exception e)` por `catch (IOException e)` en `LibroService.java:50`** (COD-FIAB-01) y agregar `log.error(...)` para registrar el fallo.
11. **Cambiar la importación de `Transactional` en `UsuarioService.java:16` a `org.springframework.transaction.annotation.Transactional`** para habilitar atributos de Spring como `rollbackFor`.

---

## 6. Conclusiones y Próximos Pasos

El Sistema de Gestión Bibliotecaria del Equipo 58-1 **no satisface** los requisitos de fiabilidad estática definidos en la característica *Reliability* de ISO/IEC 25010. La ausencia de un manejador global de

excepciones en el backend y la falta de atomicidad transaccional en los flujos críticos de préstamos constituyen los hallazgos de mayor severidad, con impacto directo sobre la consistencia de los datos y la seguridad de la información expuesta al cliente.

## Acciones inmediatas requeridas

- **Bloquear** la transición a pruebas dinámicas hasta remediar los defectos CRITICAL.
- **Priorizar** los tres defectos CRITICAL antes de la próxima entrega.
- **Repetir** la auditoría estática completa una vez aplicadas las correcciones, a fin de validar el cierre del Quality Gate.

## Próxima iteración de auditoría

Se recomienda re-ejecutar la presente checklist una vez remediados los defectos HIGH, registrando los resultados en un nuevo informe que referencie este documento como línea base. Los ítems calificados como "No Aplica" deberán re-evaluarse si en el futuro la aplicación incorpora clientes HTTP salientes, tareas programadas o acoplamientos con servicios remotos.

---

### Fin del informe.

*Documento generado el 2 de junio de 2026 por el Equipo SQA 11 como parte del proceso de inspección estática bajo ISO/IEC/IEEE 29119.*